

Learning Expectations in Dynare

Social Learning and Adaptive Learning

Status. Working document and explanation on how to use the toolbox learning

References. Social learning: Grimaud, Salle & Vermandel, “A Dynare Toolbox for Social Learning Expectations,” *Journal of Economic Dynamics and Control*, 2025. Adaptive learning: Slobodyan & Wouters, “Learning in a Medium-Scale DSGE Model with Expectations Based on Small Forecasting Models,” *American Economic Journal: Macroeconomics*, 4(2), 2012.

Contents

1	Overview	3
2	Model declaration	3
2.1	SL_-prefixed parameters	3
2.1.1	Naming convention	3
2.1.2	The SL_active switch	4
2.1.3	Example	5
2.2	Belief shocks	5
2.2.1	The selection matrices 0r and 0x	5
2.2.2	The shocks block	5
3	Stochastic simulation under Social Learning	6
3.1	Standard Dynare options	6
3.2	Fields of options_learning.social	7
3.3	Option scanning and rejected configurations	7
3.4	Random number streams	8
3.5	Computing the R matrix (compute_ghr)	8
3.6	Forward simulation (simult_SL)	8
3.7	Conventions that are not in the papers	9
3.8	Moments	9
3.9	Impulse response functions	10
3.10	Historical decomposition	10
4	Estimation under Social Learning	10
4.1	Setting up estimation	10
4.1.1	Priors	11
4.1.2	One-off setup	11
4.2	The inversion filter (IF_SL)	11
4.3	Log-likelihood (dsge_likelihood_SL)	12
4.4	Smoother output	12
5	Adaptive learning	13
6	Limitations and rejected configurations	13

7	Stored results reference	14
7.1	oo_fields	14
7.2	options_.learning.social fields	15
A	Internal architecture	16
A.1	Package map	16
A.2	Entry-point convention	17
A.3	Hook points in Dynare core	17
A.4	Test suites	18

1 Overview

The `matlab/+learning` package replaces the rational-expectations (RE) assumption by a mechanism in which agents form expectations from something other than the model solution. Two mechanisms are implemented:

- **Social learning** (`+learning/+social`). A population of agents holds heterogeneous beliefs about the forward-looking variables. Each period a fraction of them receives news, beliefs are ranked by a discounted forecast-error criterion, and pairs of agents adopt the fitter of the two. The aggregate belief wedge enters the policy function through an additional matrix R . Simulation and estimation are both supported.
- **Adaptive learning** (`+learning/+adaptive`). Agents forecast the forward variables with a small perceived law of motion whose coefficients are updated by constant-gain recursive least squares. The actual law of motion is re-solved every period from those beliefs. Simulation only.

Both mechanisms are configured through *naming conventions* on parameters declared in the `.mod` file. The C++ preprocessor is not modified: at runtime, `setup_options_SL` (resp. `setup_options_AL`) scans `M_.param_names` and `M_.exo_names` and populates `options_.learning.social` (resp. `options_.learning.adaptive`). If no learning parameter is found, the function returns silently and Dynare proceeds in standard RE mode.

Scope of this document. Sections 2–4 describe social learning, which is the more complete of the two. Section 5 describes adaptive learning. Section 6 lists what is not supported, and section A documents the internal architecture.

2 Model declaration

Declaring a social learning model requires three things in the `.mod` file:

1. **A belief shock per learned variable.** For each forward-looking variable that agents learn about, an exogenous variable `a_VARNAME` is declared in `varexo` (section 2.2).
2. **SL_-prefixed parameters.** The hyperparameters governing the learning dynamics are declared as ordinary Dynare `parameters` with names starting with `SL_` (section 2.1).
3. **A shocks block.** Structural shocks get their usual variance. The belief shocks `a_VARNAME` must keep a zero variance for simulation and get a positive, estimated variance for estimation (section 2.2.2).

2.1 SL_-prefixed parameters

2.1.1 Naming convention

Parameter in <code>.mod</code>	Field in <code>options_.learning.social</code>	Default	Description
<code>SL_active</code>	--	1	switch on/off du learning, simulation path only
<code>SL_n_agents</code>	<code>N_agents</code>	300	Population size J

Parameter in <code>.mod</code>	Field in <code>options_.learning.social</code>	Default	Description
<code>SL_wedding_proportion</code>	<code>wed_p</code>	1.0	Share of the population entering the tournament
<code>SL_t_max</code>	<code>t_max</code>	1	Horizon of the reported h -step-ahead expectations
<code>SL_mutation_prob_{VAR}</code>	<code>mut_p</code>	0.3	News probability μ for VAR
<code>SL_mutation_std_VAR</code>	<code>mut_sd(iv)</code>	—	Belief dispersion ξ_v for VAR. Declaring it with a strictly positive value is what makes VAR a learned variable
<code>SL_fitness_discount_VAR</code>	<code>rhoGN(iv)</code>	—	Fitness memory δ_v , must lie in $(0, 1)$
<code>SL_mutation_prob_VAR</code>	<code>mut_p(iv)</code>	global	Per-variable override of μ

The last four rows are *per-variable*: the suffix `VAR` must match a forward-looking endogenous variable. This is how the scan knows which variables are learned. Declaring `SL_mutation_std_pi` makes the scan look for the endogenous variable `pi` in the forward-looking set and for the shock `a_pi` in `varexo`; if either is missing, an error is raised.

`SL_fitness_discount_VAR` has no default: a learned variable whose discount is missing or outside $(0, 1)$ raises `learning:social:badDiscount`. A zero discount makes the fitness identically zero, so the tournament degenerates into an unconditional copy and beliefs random-walk with no selection.

The scan stores the resolved configuration in two places:

- `M_.params` holds the raw numerical values, as written by the preprocessor and updated by `set_all_parameters` at each MCMC iteration;
- `options_.learning.social` holds the runtime configuration consumed by the simulation and estimation routines, rebuilt from `M_.params` by `setup_options_SL`.

The SL hyperparameters can be estimated by Bayesian MCMC like any other parameter: they can be found in `estimated_params`, have priors, and at each likelihood evaluation `refresh_learning_params` re-reads `M_.params` into `options_.learning.social` (section 4.3).

2.1.2 The `SL_active` switch

When `SL_active` is declared and set to 0, `options_.learning` is not built at all and Dynare stays on the native RE path. A warning is issued and a second warning if a belief shock still carries a non-zero variance, since that configuration is “RE plus an expectation shock” rather than pure RE.

The switch applies to **simulation only**. The estimation path calls `setup_options_SL` with `force_active = true`, which bypasses it to avoid confusion in estimation.

2.1.3 Example

Listing 1: SL parameters in the .mod file

```
parameters beta kappa sigma phi_pi phi_y rho_mp // structural
           SL_active SL_n_agents
           SL_mutation_std_pi SL_fitness_discount_pi SL_mutation_prob_pi ; // SL per-
           variable

beta = 0.99; kappa = 0.03; sigma = 1;
phi_pi = 1.50; phi_y = 0.125; rho_mp = 0;

SL_active = 1;
SL_n_agents = 300;
SL_mutation_prob_pi = 0.3;
SL_mutation_std_pi = 0.003;
SL_fitness_discount_pi = 0.5;
```

2.2 Belief shocks

A learned variable enters the model through an expectation equation of the form

```
pie = pi(+1) + a_pi;
```

where `pie` is the subjective expectation of next-period inflation. The `(+1)` term is the model-consistent component and `a_pi` is the belief wedge.

From Dynare's point of view `a_pi` is an ordinary exogenous variable. During simulation, however, its realisation is not drawn from a shock distribution: the wedge is produced by the learning algorithm itself. The naming convention `a_VARNAME` is what links the shock to the learned variable and builds the selection matrix `Or`.

2.2.1 The selection matrices `Or` and `Ox`

The scan produces two matrices in `options_.learning.social`:

- `Or` ($n_{\text{fwd}} \times n_{\text{exo}}$): entry $(i, j) = 1$ if the i -th forward variable is learned and the j -th exogenous is its belief channel. Each learned variable has exactly one channel and each channel feeds exactly one variable; both are checked.
- `Ox` ($n_{\text{exo}} \times n_{\text{exo}}$): the complementary projector $I - O_r' O_r$, which selects the structural shocks.

2.2.2 The shocks block

The variance assigned to a belief shock means different things in the two contexts.

Simulation. Leave `a_pi` out of the `shocks` block, or set `var a_pi = 0`. The belief wedge φ_t is generated entirely by mutation and the tournament. A non-zero variance turns the shock into an ordinary aggregate news shock.

Estimation. Give `stderr a_pi` a positive value and estimate it. The inversion filter requires as many shocks as observables, so the belief channel carries part of the identification; its standard deviation is an estimated parameter.

3 Stochastic simulation under Social Learning

When `options_.learning.social` is populated, `stoch_simul` activates the SL pipeline. The RE decision rules are solved first, then the R matrix is computed, then the simulation runs forward with heterogeneous beliefs.

1. `learning.social.setup_options_SL` — scan the parameters, build `options_.learning.social`, reject unsupported configurations (section 3.3).
2. `learning.social.compute_ghr` — compute R from the dynamic Jacobian (section 3.5).
3. `learning.social.simult_SL` — forward simulation with mutation and tournament (section 3.6).
4. `learning.social.moments_SL` and `learning.social.irf_SL` — Monte Carlo moments and generalised IRFs (sections 3.8 and 3.9).

3.1 Standard Dynare options

The SL path reuses the native `stoch_simul` options rather than introducing parallel ones.

order *INTEGER* Order of the Taylor approximation. Social learning requires `order=1`; a higher order raises `learning:social:order`.

Default: 1.

periods *INTEGER* Length of each simulated path. Required: SL moments are always simulated, never analytical. `periods - drop` must be at least 100.

Default: 0.

drop *INTEGER* Burn-in discarded before moments are measured, and common burn-in before the impulse in the generalised IRFs.

Default: 100.

replic *INTEGER* Number of Monte Carlo replications over agent draws, for both moments and IRFs. This is what the empirical confidence bands are computed from. A value of 1 produces no bands and raises a warning.

Default: 1.

irf *INTEGER* Horizon of the generalised IRFs.

Default: 40.

ar *INTEGER* Number of autocorrelation lags reported.

Default: 5.

noprint *BOOLEAN* Suppress the summary tables.

Default: false.

The random number generator matters here. The learning streams require an algorithm that supports substreams; declaring `set_dynare_seed('mrg32k3a', N)` in the `.mod` makes the run reproducible and silences the promotion notice (section 3.4).

3.2 Fields of `options_.learning.social`

These are populated automatically from the `SL_`-prefixed parameters. They can be overridden by hand after `setup_options_SL` has run, which is what the test suite does.

N_agents *INTEGER* Population size J . The tournament pairs agents two by two, so an odd value is rounded up with a warning.

Default: 300.

mut_p *DOUBLE* $[n_{fwd} \times 1]$ News probability μ . The global value fills every forward row, then the per-variable overrides apply on top. Must lie in $(0, 1]$: the belief loop draws $\text{round}(\mu_v J)$ mutants and rescales the news by $1/\mu_v$ (section 3.7).

Default: 0.3.

wed_p *DOUBLE* Share of the population entering the tournament. The number of paired agents is $2\lfloor \text{wed_p} \cdot J/2 \rfloor$.

Default: 1.0.

mut_sd *DOUBLE* $[n_{fwd} \times 1]$ Belief dispersion ξ . Entry i is non-zero only if the i -th forward variable is learned.

Default: 0.

rhoGN *DOUBLE* $[n_{fwd} \times 1]$ Fitness memory δ , in $(0, 1)$ for every learned variable.

Default: 0.

t_max *INTEGER* Horizon of the h -step-ahead aggregate expectations reported in `expectations_ahead`. Diagnostic output only: it does not affect the simulated path.

Default: 1.

decomp *BOOLEAN* Compute the historical decomposition of the simulated path into structural shock channels and belief channels (section 3.10).

Default: false.

irf_quantiles *DOUBLE vector* Quantile levels of the generalised IRF bands.

Default: [.05 .5 .95].

initial_beliefs *DOUBLE* $[n_{fwd} \times J]$ Initial belief array.

Default: 0.

3.3 Option scanning and rejected configurations

`setup_options_SL` is called at the very beginning of `stoch_simul`. Beyond building the option structure, it validates the declaration: mutation probabilities in range, discounts in $(0, 1)$, a one-to-one mapping between learned variables and belief shocks, and O_x idempotent. Every failure raises an error with an identifier prefixed `learning:social:`.

It also calls `learning.social.check_support`, which rejects the configurations listed in section 6.

3.4 Random number streams

Social learning uses random streams for two purposes: the belief path (which agents mutate, how they are paired) and the structural shocks. The two are drawn from *separate, non-overlapping substreams* of a single seed, built by `learning.learning_streams`:

- **'sim'** mode, replication k : beliefs on substream $2k - 1$, shocks on substream $2k$. Replications are therefore independent and **parfor**-safe.
- **'estim'** mode: beliefs on a *fixed* substream, so that the belief realisation is frozen across parameter draws. This is what makes the likelihood a deterministic function of θ and the posterior surface smooth enough for a mode-finder and for MCMC.

The stream is never derived from `options_` inside `simult_SL` or `IF_SL`: it is passed as an explicit argument, and both functions raise an error if it is missing. Common random numbers therefore stay under the caller's control.

Substreams require `mrg32k3a` or `mlfg6331_64`. If the declared algorithm has none, the learning streams are built on `mrg32k3a` anyway and a notice is printed once.

3.5 Computing the R matrix (`compute_ghr`)

Under social learning the policy function is

$$y_t = \bar{y} + P(y_{t-1} - \bar{y}) + Q O_x \varepsilon_t + R_f(\varphi_t + O_r \varepsilon_t), \quad (1)$$

where P and Q are the RE decision-rule matrices (`ghx`, `ghu` re-indexed in declaration order), R_f collects the forward-variable columns of R , and φ_t is the aggregate belief wedge. The O_x/O_r split is what separates the structural shocks from the belief channel: a given exogenous feeds either the state through Q or the beliefs through R_f , never both.

Writing the model as $F E_t y_{t+1} + G y_t + H y_{t-1} + M \varepsilon_t = 0$ and assuming that agents apply the policy function and add their own wedge on top, $E_t y_{t+1} = P y_t + R \varphi_t + \varphi_t$, gives

$$R = -(I + A^{-1}F)^{-1} A^{-1}F, \quad A \equiv FP + G. \quad (2)$$

R is $n_{\text{endo}} \times n_{\text{endo}}$ but only the forward-variable columns are non-zero. It is computed once after `resol` and stored in `oo_.dr.ghr`, leaving `ghx`, `ghu` and `ys` untouched.

Consistency check. On every call, the first-order solution is validated against the Jacobian: $P = -A^{-1}H$ and $Q = -A^{-1}M$ must hold to 10^{-8} in relative terms. A failure means `oo_.dr` and `M_` do not describe the same model at the same point — stale decision rules, wrong parameters, or an approximation order the derivation does not cover. On a healthy model the residuals sit at 10^{-15} .

3.6 Forward simulation (`simult_SL`)

Algorithm. At each period t , in this order:

1. **Mutation.** For each forward variable v , draw $\text{round}(\mu_v J)$ agents uniformly — one independent `randperm` per variable — and perturb their belief by $\mathcal{N}(0, \xi_v^2)$. This gives the candidate beliefs $m_{j,t}$.
2. **Aggregation and transition.** The candidate aggregate $\varphi_t = \frac{1}{J} \sum_j m_{j,t}$ enters the policy function (1), producing y_t .
3. **News injection.** The realisation of the belief shock is added to the mutants' beliefs, rescaled by $1/\mu_v$ (section 3.7).

4. **Fitness.** Each agent's belief is scored by a discounted sum of squared forecast errors, the forecast being the one implied by the agent's own belief and the policy function.
5. **Tournament.** Agents are paired at random; both members of a pair adopt the belief of the fitter one. This gives the post-tournament beliefs $a_{j,t}$, which are the starting point of period $t + 1$.

Note that aggregation precedes the fitness evaluation: beliefs influence y_t , and are then scored against that same y_t .

Outputs.

- `oo_.endo_simul` — simulated path, $n_{\text{endo}} \times T$ (no initial-condition column).
- `oo_.learning.social.beliefs` — post-tournament beliefs $a_{j,t}$, $n_{\text{fwr}} \times J \times T$.
- `oo_.learning.social.beliefs_candidate` — post-injection candidates $m_{j,t}$, same size.
- `oo_.learning.social.fitness` — fitness values, same size.
- `oo_.learning.social.expectations` — aggregate wedge entering the policy function, $\varphi_t + O_r \varepsilon_t$, $n_{\text{fwr}} \times T$.
- `oo_.learning.social.expectations_ahead` — h -step-ahead aggregate expectations, $n_{\text{fwr}} \times T \times \text{t_max}$. Diagnostic only.
- `oo_.learning.social.final_beliefs` — beliefs at $t = T$, $n_{\text{fwr}} \times J$.
- `oo_.learning.social.y_RE` — the RE trajectory under the same structural shocks, for comparison.

3.7 Conventions that are not in the papers

Some implementation choices are not derivable from the published description and matter for anyone reading or extending the code.

News rescaling by $1/\mu$. The aggregate news $O_r \varepsilon_t$ is injected only into the beliefs of the mutating agents, divided by μ_v , so that the population average moves by exactly $O_r \varepsilon_t$. This holds exactly when $\text{round}(\mu_v J) = \mu_v J$ and approximately otherwise. It is also why $\mu_v = 0$ is forbidden.

3.8 Moments

`learning.social.moments_SL` runs `replic` independent simulations — each on its own pair of substreams — and averages the resulting moments. Every replication starts agents at zero beliefs and discards `drop` burn-in periods, so that the cross-sectional distribution of beliefs differs across replications at the start of the measurement window.

Outputs.

- `oo_.mean`, `oo_.var`, `oo_.autocorr{1:ar}` — Monte Carlo averages, in the standard Dynare format but always simulated.
- `oo_.learning.social.moments.mean_ci`, `.std_ci`, `.autocorr_ci{1:ar}` — empirical 2.5/97.5 percentile intervals across replications.

- `oo_.learning.social.moments.cross_section` — cross-sectional belief statistics: `std_mean`, `iqr_mean`, `skew_mean` averaged over time and replications, plus the time-series versions `*_ts`.
- `oo_.learning.social.moments.mc_raw` — the raw replication-level data.

Display. `disp_moments_SL` prints three tables: aggregate moments with their intervals against the RE theoretical standard deviations, first-order autocorrelations against their RE counterparts, and the cross-sectional belief moments. The RE column comes from the native `th_autocovariances`, so it honours `hp_filter` and `bandpass` while the SL column does not; a warning is issued when either filter is active.

3.9 Impulse response functions

`learning.social.irf_SL` computes generalised IRFs by paired Monte Carlo. For each replication, two simulations are run from the same belief substream — the stream state is saved and restored between them — one baseline and one with an impulse at the first post-burn-in period. The GIRF is the difference, and the reported response is the median across replications.

The impulse is the corresponding column of the lower Cholesky factor, $cs(:, j)$, exactly as in the native `irf` path, so the responses are orthogonalised and comparable with the RE benchmark.

Outputs. For each shock, `oo_.learning.social.irf.shock` holds `median`, `mean` and `bands`. The median GIRF is also written to the standard `oo_.irfs` structure, so Dynare’s plotting routines work unchanged.

3.10 Historical decomposition

With `options_.learning.social.decomp` set, `simult_SL` also propagates one trajectory per channel: one per structural shock and one per forward variable for the belief channels. The contributions sum to $y_t - \bar{y}$ by construction.

`oo_.learning.social.decomp` is $n_{\text{endo}} \times T \times n_{\text{chan}}$, aligned period by period on `endo_simul`, with `decomp_names` giving the channel labels. `learning.social.plot_decomp` draws the stacked contributions with the total trajectory overlaid.

The adding-up is exact only when $\text{round}(\mu_v J) = \mu_v J$, for the reason given in section 3.7; the residual is reported.

4 Estimation under Social Learning

The SL likelihood plugs into Dynare’s native Bayesian estimation infrastructure. All standard `estimation` options, the mode-finders, the MCMC machinery and the posterior diagnostics are available.

4.1 Setting up estimation

Data are declared exactly as in a standard estimation. The binding constraint is the inversion identification condition: **the number of observables must equal the number of exogenous shocks**, belief shocks included. This is the same requirement as Dynare’s `conditional_likelihood`, and it also implies no measurement error and no missing observations.

4.1.1 Priors

Listing 2: Priors for SL parameters

```
estimated_params;
// structural
rho_d, , 0, 0.999, beta_pdf, 0.5, 0.2;
kappa, , 0, 0.999, gamma_pdf, 0.3, 0.05;
phi_pi, , 1, 10.00, gamma_pdf, 1.5, 0.1;
// shock standard deviations, including the belief channel
stderr e_d, , 0, 1, inv_gamma2_pdf, 0.001, 1;
stderr e_s, , 0, 1, inv_gamma2_pdf, 0.001, 1;
stderr a_pi, , 0, 1, inv_gamma2_pdf, 0.001, 1;
// SL hyperparameters
SL_mutation_std_pi, , 0, 0.999, inv_gamma_pdf, 0.003, 1;
SL_fitness_discount_pi, , 0, 0.999, beta_pdf, 0.8, 0.1;
SL_mutation_prob, , 0, 0.999, beta_pdf, 0.3, 0.05;
end;
```

Bound μ and δ away from the edges of $(0, 1)$. A draw outside the admissible range is rejected with an infinite objective rather than crashing, but a prior that puts mass there wastes evaluations.

4.1.2 One-off setup

Before the likelihood loop, `setup_learning_estimation` runs once — the SL counterpart of `dynare_estimation_init`. It builds the observation selection matrix O , the forward-variable indices, and the index maps linking positions in `M_.params` to the `mut_sd`, `rhoGN` and `mut_p` vectors. These maps let `refresh_learning_params` inject the current draw at each evaluation without re-scanning `M_.param_names`.

4.2 The inversion filter (IF_SL)

Given a parameter vector θ and the solved model:

1. Initialise J agents at zero beliefs and the state at its steady state.
2. For each period t :
 - (a) run the mutation step to obtain the candidate beliefs and the aggregate φ_t ;
 - (b) invert the observation equation to extract the shocks, $\hat{\varepsilon}_t = \lambda^{-1}(y_t^{\text{obs}} - c - O(P\hat{y}_{t-1} + R_f\varphi_t))$;
 - (c) feed $\hat{\varepsilon}_t$ into the transition and into the belief recursion, then run the fitness and tournament steps.
3. Accumulate the per-period log-likelihood contributions.

The inversion Jacobian is $\lambda = OQO_x + OR_fO_r$, constant across periods. It is checked to be well conditioned: an ill-conditioned λ means the observables carry almost no independent information about the shocks.

Measurement constant. The data are compared to $c + O(y_t - \bar{y})$ with $c = O\bar{y}$, or $c = 0$ when `prefilter` is set and the data have been demeaned. The constant that was removed is reported in `oo_.Smoother.Constant`, following the native convention.

Random numbers. IF_SL does *not* build its own stream. The belief stream is passed as the fifth argument, and the function raises `learning:social:noStream` if it is missing. The caller (`dsge_likelihood_SL`) builds it in 'estim' mode at every evaluation, which is what freezes the belief realisation across parameter draws.

Early stopping. If the cumulated inversion residual exceeds a threshold, the loop stops and reports how many periods were left unprocessed. The likelihood is then penalised, and the smoother marks the unreached periods as NaN rather than leaving them at their initial value.

4.3 Log-likelihood (`dsge_likelihood_SL`)

`learning.social.dsge_likelihood_SL` has the same argument and output signature as the native `dsge_likelihood`, so it plugs into the mode-finders and the samplers without modification.

Formula. With n_e shocks of positive variance and T observations after the presample,

$$\log \mathcal{L}(\theta) = -\frac{n_e T}{2} \log(2\pi) - \frac{T}{2} \log|\Sigma_\varepsilon| - \frac{1}{2} \sum_t \hat{\varepsilon}_t' \Sigma_\varepsilon^{-1} \hat{\varepsilon}_t - T \log|\det \lambda|. \quad (3)$$

The last term is the Jacobian of the change of variables from observables to shocks: it is what distinguishes an inversion filter from a plain Gaussian quadratic form. Both determinants are evaluated through factorisations (Cholesky for Σ_ε , LU for λ) rather than `det`.

Parameter refresh. At each evaluation, `refresh_learning_params` copies the current `M_.params` into the SL option fields using the precomputed index maps. The idiom `L.mut_p(:) = scalar` is load-bearing: without the `(:)`, the $[n_{\text{fwr}} \times 1]$ vector would collapse to a scalar and silently break the per-variable logic. The refreshed values are then range-checked, and a draw outside the admissible set returns an infinite objective rather than propagating into the belief loop.

4.4 Smoother output

`store_SL_smoother` routes the inversion filter output into `oo_`:

- `oo_.SmoothedShocks.exo` — smoothed shocks, belief channels included.
- `oo_.SmoothedVariables.endo` — smoothed state trajectory.
- `oo_.Smoother.SteadyState`, `oo_.Smoother.Constant` — for reporting compatibility.
- `oo_.learning.social.smoother` — the SL-specific payload: `expectations` ($T \times n_{\text{fwr}}$, the aggregate belief wedge, the same object as `oo_.learning.social.expectations` on the simulation path), `forward_states` (the realised forward variables), `forward_vars`, and the per-shock news series.

Periods the inversion did not reach are NaN in every output.

`plot_SL_smoother_results` produces four figures: smoothed structural shocks; smoothed observables overlaid on the data; the aggregate news channel; and the aggregate belief wedge on the learned variables, which is the deviation of subjective expectations from their rational counterpart.

5 Adaptive learning

The `+learning/+adaptive` package implements constant-gain adaptive learning. It is a simulation proof of concept to show you can add your own learning type with only a few functions and hooks in Dynare. For now there is no likelihood and no estimation path.

Reference and relation to the reference. The algebra and MATLAB implementation of this package were inspired by the article and replication code of Slobodyan and Wouters (2012). In their framework, beliefs are updated using a Kalman filter, while the constant-gain specification is considered only in their sensitivity analysis. This package implements this constant-gain variant. Their projection facility, which prevents belief updates from destabilising the actual law of motion, is also reflected in `simultAL`: *whenthere—solvedtransitionmatrixhasaspectralradiusabovethespec*

Mechanism. Agents forecast the forward variables with a small perceived law of motion (PLM), either an intercept only or an intercept plus an AR(1) slope, estimated variable by variable. Every period the coefficients are updated by constant-gain recursive least squares, and the actual law of motion is re-solved from the updated beliefs, so the transition matrices (μ_t, T_t, R_t) move over time.

Declaration. No belief shock is needed: beliefs enter by re-solving the model, not through an exogenous channel. Only parameters are declared.

Parameter	Field	Default	Description
AL_active	--	0	Master switch
AL_gain	gain	0.02	Constant gain of the RLS updating
AL_pj_threshold	pj_threshold	1.0	Projection facility threshold on $\max \text{eig}(T) $
AL_plm_type	plm_type	1	1 = intercept + AR(1), 0 = intercept only
AL_learn_VAR	id_fwrd	all	Learn VAR or not; if none is declared, all forward variables are learned

Outputs. `oo.endo_simul` plus `oo.learning.adaptive.info` with `beta_hist` ($n_{\text{fwr}} \times n_b \times T$, the belief path), `beta_final` and `pf_hits` (the number of rejected updates). Monte Carlo moments are produced by `moments_AL` in the same format as the SL ones, with the belief-coefficient dynamics replacing the cross-sectional statistics.

Not implemented. Impulse responses: `stoch_simul` warns and falls back to the RE responses, which ignore the learning dynamics. Estimation. Richer PLMs (full state, AR(2)). Kalman filter learning.

6 Limitations and rejected configurations

`check_support` is called on both the simulation and the estimation path and raises an error for the configurations below, rather than letting them run to completion and return wrong results.

Configuration	Reason
order > 1	The expectations matrix is derived from the first-order solution.
loglinear	The Jacobian is evaluated at oo.dr.ys, which loglinear replaces by its logarithm.
Deterministic exogenous variables	The Jacobian slicing does not cover them.
Measurement errors (estimated or calibrated)	The inversion filter requires as many shocks as observables, with no measurement noise.
Estimated shock correlations	The O_x/O_r split assumes news and structural shocks are orthogonal.
Skewed shocks	The likelihood assumes Gaussian innovations.
Deterministic trends in the measurement equation	Not handled by the inversion.
Diffuse and heteroskedastic filters	Not applicable; Σ_ϵ is assumed constant.
forecast	The smoother does not populate the oo.Smoother fields forecasts.run requires.
histval	The simulation always starts from the steady state.

A second group produces a warning rather than an error, because the run is valid but an output is silently missing or not comparable: `filtered_vars`, `filter_covariance`, `filter_decomposition`, `moments_varendo` and `bayesian_irf` rely on Kalman-filter objects the inversion does not produce; `hp_filter` and `bandpass` are honoured by the RE column of the moment tables but not by the simulated SL column.

`prefilter` is supported: the measurement constant is dropped when the data have been demeaned.

Octave. The learning streams require MATLAB RandStream substreams, which have no Octave equivalent, and both `simult_SL` and `IF_SL` reject a non-RandStream argument. The package therefore does not run under Octave at all.

7 Stored results reference

7.1 oo_fields

Field	Description
oo.dr.ghr	R matrix, $n_{\text{endo}} \times n_{\text{endo}}$
<i>Simulation</i>	
oo.endo_simul	Simulated path, $n_{\text{endo}} \times T$
oo.learning.social.beliefs	Post-tournament beliefs, $n_{\text{fwr}} \times J \times T$
oo.learning.social.beliefs_candidates	Post-injection candidates, same size
oo.learning.social.fitness	Fitness values, same size
oo.learning.social.expectations	Aggregate wedge $\varphi_t + O_r \varepsilon_t$, $n_{\text{fwr}} \times T$

Field	Description
oo_.learning.social.expectations_ahead	h-step-ahead expectations, $n_{\text{fwr}} \times T \times t_{\text{max}}$
oo_.learning.social.final_beliefs	Beliefs at $t = T$, $n_{\text{fwr}} \times J$
oo_.learning.social.y_RE	RE path under the same structural shocks
oo_.learning.social.decomp	Historical decomposition, $n_{\text{endo}} \times T \times n_{\text{chan}}$
<i>Moments and IRFs</i>	
oo_.mean, oo_.var, oo_.autocorr	Monte Carlo moments
oo_.learning.social.moments.mean_ci	Empirical intervals on the means
oo_.learning.social.moments.std_ci	Empirical intervals on the standard deviations
oo_.learning.social.moments.cross_section	Cross-sectional belief statistics
oo_.learning.social.moments.mc_raw	Replication-level raw data
oo_.learning.social.irf.shock	.median, .mean, .bands
<i>Estimation</i>	
oo_.SmoothedShocks	Smoothed shocks, belief channels included
oo_.SmoothedVariables	Smoothed states
oo_.Smoother.Constant	Constant removed from the measurement equation
oo_.learning.social.smoother	.expectations, .forward_states, .forward_vars, .news
<i>Adaptive learning</i>	
oo_.learning.adaptive.info	.beta_hist, .beta_final, .pf_hits
oo_.learning.adaptive.moments	Same structure as the SL moments

7.2 options_.learning.social fields

Field	Type	Description
N_agents	integer	Population size J
mut_p	double [nfwr $\times$ 1]	News probability μ
wed_p	double	Tournament participation share
mut_sd	double [nfwr $\times$ 1]	Belief dispersion ξ
rhoGN	double [nfwr $\times$ 1]	Fitness memory δ
t_max	integer	Horizon of the reported expectations
Or	double [nfwr $\times$ nexo]	Belief channel selection
Ox	double [nexo $\times$ nexo]	Structural shock projector $I - O_r' O_r$
id_SL	integer vector	Indices of the learned variables
n_SL	integer	Number of learned variables

Field	Type	Description
decomp	logical	Compute the historical decomposition
irf_quantiles	double vector	GIRF band levels
<i>Added by setup_learning_estimation</i>		
0	double [nobs×nendo]	Observation selection
id_obs	integer vector	Indices of the observed variables
id_fwrd	integer vector	Indices of the forward variables
nfwr	integer	Number of forward variables
omega	double [nfwr×nendo]	Forward selection
mut_pid, mut_vid	integer vectors	Parameter and variable index maps for ξ
rhoGN_pid, rhoGN_vid	integer vectors	Idem for δ
mutp_pid, mutp_vid	integer vectors	Idem for the per-variable μ
mut_freq_pid	integer	Index of the global SL_mutation_prob
estimation	logical	Set when the estimation objects are built

A Internal architecture

A.1 Package map

+learning holds what is shared between mechanisms, and one sub-package per mechanism.

File	Role
<i>+learning</i>	
learning_streams.m	Build the belief and shock substreams from one seed
is_active.m	True when a learning mechanism is configured
getopt.m	Read an optional option field with a default
<i>+learning/+social — simulation</i>	
setup_options_SL.m	Build options_.learning.social from the SL_* parameters
check_support.m	Reject or flag unsupported configurations
compute_ghr.m	Compute and validate the R matrix
simult_SL.m	Forward simulation with mutation and tournament
moments_SL.m	Monte Carlo moments with empirical intervals
disp_moments_SL.m	Moment tables
irf_SL.m	Paired Monte Carlo generalised IRFs
plot_decomp.m	Historical decomposition plots

File	Role
<i>+learning/+social — estimation</i>	
setup_learning_estimation.m	Build the θ -invariant estimation objects, run once
refresh_learning_params.m	Inject the current draw, hot path
dsge_likelihood_SL.m	Log-posterior, native signature
IF_SL.m	Inversion filter core
store_SL_smoother.m	Route the smoother payload into oo_
plot_SL_smoother_results.m	Post-estimation figures
<i>+learning/+adaptive</i>	
setup_options_AL.m	Build options_.learning.adaptive
check_support_AL.m	Rejected configurations
init_beliefs_AL.m	Model-consistent initial beliefs
update_beliefs_AL.m	Constant-gain RLS updating
resolve_alm_AL.m	Map beliefs into the actual law of motion
simult_AL.m	Forward simulation with belief updating
moments_AL.m	Monte Carlo moments
disp_moments_AL.m	Moment tables

A.2 Entry-point convention

Every mechanism entry point takes (M_, options_, oo_, ...) and returns oo_:

```
oo_ = learning.social.simult_SL(M_, options_, oo_, ee, belief_rng);
oo_ = learning.adaptive.simult_AL(M_, options_, oo_, ee, belief_rng);
oo_ = learning.social.moments_SL(M_, options_, oo_);
oo_ = learning.adaptive.moments_AL(M_, options_, oo_);
```

Shock matrices are [T x nexo] in both packages. simult_AL accepts a stream argument it does not use, so that the two mechanisms share one signature.

A.3 Hook points in Dynare core

Two core files are modified.

stoch_simul.m -- five points: the option scan at the top; compute_ghr after resol; the simulation block; the moments block; and the IRF loop. The shock draw is shared between mechanisms, only the final dispatch differs:

```
if learning.is_active(options_)
    id_struct = find(diag(M_.Sigma_e) > 0);
    cs = get_lower_cholesky_covariance(M_.Sigma_e, ...
        options_.add_tiny_number_to_cholesky);
    chol_struct = cs(id_struct, id_struct)';
    [belief_rng, shock_rng] = learning.learning_streams( ...
        options_.DynareRandomStreams.seed, ...
        options_.DynareRandomStreams.algo, 'sim', 1);
    ee = zeros(options_.periods, M_.exo_nbr);
    ee(:, id_struct) = randn(shock_rng, options_.periods, ...
        numel(id_struct)) * chol_struct;
    if learning.is_active(options_, 'social')
        oo_ = learning.social.simult_SL(M_, options_, oo_, ee, belief_rng);
    else
```

```

        oo_ = learning.adaptive.simult_AL(M_, options_, oo_, ee, belief_rng);
    end
end

```

`dynare_estimation_1.m` -- three points: the choice of the objective function, and the two smoother branches.

```

% objective function
elseif learning.is_active(options_, 'social') && ...
    learning.getopt(options_.learning.social, 'estimation', false)
    objective_function = str2func('learning.social.dsge_likelihood_SL');

```

Dynare reaches the classical smoother through different point in the code : `options_.frequentist_smoother` which defaults to true, and not only through `options_.smoother`. A branch gated on `options_.smoother` alone is skipped without any message, and the run silently produces Kalman-filtered variables from the RE solution. Both flags must be tested.

```

if learning.is_active(options_, 'social') && ...
    (learning.getopt(options_, 'smoother', false) || ...
    learning.getopt(options_, 'frequentist_smoother', false))
    [~,~,~,~,~,model_solution] = learning.social.dsge_likelihood_SL(...);
    oo_ = learning.social.store_SL_smoother(oo_, M_, options_, model_solution);
    learning.social.plot_SL_smoother_results(oo_, M_, options_, dataset_);
elseif options_.frequentist_smoother
    oo_ = save_display_classical_smoother_results(...);
end

```

This is one of weakest part of the integration: Dynare has several entry points to the smoother, guarded by composite conditions on `mode_compute`, `mode_file`, `mh_replic` and `load_mh_file`. Covering each of them by a separate hook is not robust. A single branching point would be preferable.

A.4 Test suites

`tests/learning/social` holds seven `.mod` files, each one a Dynare invocation and a test, running in about eight seconds in total. `tests/learning/adaptive` holds one. Both use the shared harness in `tests/learning` and are documented in a manifest per mechanism, which lists what each check asserts, the reference values, and the known coverage gaps.

The assertions that matter most:

- **Simulation-inversion round trip.** IF_SL recovers the shocks, the state path *and* the belief path produced by `simult_SL`, to 10^{-12} , on a model with a zero steady state and on one with a non-zero steady state. The three residuals are asserted separately: merged into one norm, a convention error on the beliefs can be absorbed by the recovered shocks.
- **RE limit.** With zero belief dispersion *and* no news, the belief wedge vanishes and `simult_SL` reproduces the native `simult_` exactly; the generalised IRFs reproduce the analytic responses.
- **Deterministic likelihood.** Two evaluations at the same θ return bit-identical values. This is the only check protecting the substream architecture, and hence the validity of common random numbers.

- **Steady-state invariance.** Beliefs are invariant to a shift of the steady state while the path shifts by exactly that amount.
- **Smoother dispatch.** After an estimation, `oo_.learning.social.smoother` exists and `oo_.UpdatedVariables`, the marker of the native Kalman path, does not.
- **Input guards.** Twenty-four rejected configurations, each checked on `ME.identifier` rather than on the message text.